Hit Rate Is Not Value: When Cache Metrics Mislead in Tiered LLM KV-Cache Persistence*

Aditya Lawankar
lawankaraditya@gmail.com

Abstract

We set out to beat LRU at evicting persisted LLM KV caches with a learned policy, and failed: across three synthetic workloads, two served architectures, and a replay of the Azure LLM inference trace, LRU wins cache hit rate every time. The interesting result is what that verdict conceals. Scored by GPU time saved, the same LRU delivers the *least* value of any policy we test on enterprise traffic, where a value-density policy conceding 21 hit-rate points saves $1.99\times$ as much compute, and still $1.15\times$ once we correct a provisioning error the same analysis exposes. Hit rate assumes every hit is worth the same; KV caching violates that twice, since a hit below the break-even context length N^* is worth nothing and above it savings grow superlinearly. Both effects are closed-form in N^*, and the largest lever they expose is provisioning, not policy: deleting a tier that cannot repay its restore is worth $3.6\times$ at identical capacity. A ground-truth ablation isolates the failure: two clairvoyant policies that both cache perfectly, 100.0% against 100.0%, still deliver significantly different value.

1 Introduction

Serving a large language model proceeds in two phases: *prefill* processes the prompt in parallel and populates the KV cache, and *decode* generates one token at a time against it. Prefill costs $O(N^2)$ attention FLOPs while the cache grows only linearly in N, so the cache makes multi-turn boundaries cheap. VRAM has no room for idle sessions, though: an 80 GB A100 fits only $\sim$30 full-context Llama-2-7B sessions after weights, against thousands of users whose think time spans seconds to days. Serving engines therefore evict idle caches [6], and a returning user re-prefills everything. Tiered persistence demotes those caches down a hierarchy (VRAM $\rightarrow$ NVMe $\rightarrow$ object storage), paying $O(N)$ bytes moved instead of $O(N^2)$ FLOPs recomputed [4, 12, 10]. That relocates the eviction problem rather than eliminating it, since every tier is finite.

That decision looks like fertile ground for learning. Sessions differ by orders of magnitude in size, their recompute cost grows quadratically where their footprint grows linearly, and whether a user returns looks predictable from history. LRU sees none of this, so we hypothesized that a learned policy estimating resumption probability would beat it on hit rate. *The data falsified this hypothesis*, and diagnosing why became the contribution: LRU wins hit rate on every constrained workload, and no feature engineering closed the gap. But the metric was the problem: measured by GPU time saved, that same LRU delivers the *least* value of any policy on enterprise traffic, and policies losing to it by 11–21 hit-rate points save up to twice as much. Hit rate assumes every hit is worth the same, and KV caching breaks that twice: a hit below the break-even length N^* is worth *nothing*, and above it savings grow superlinearly (§4).

Related work. Systems that persist, disaggregate, or offload KV state across a hierarchy [6, 4, 12, 13] build the persistence mechanism; we target the policy layer above it. Prefix sharing [5, 10, 16] and

*System, committed run records, and the scripts regenerating every number, figure, and table here: `https://github.com/aditya-lawankar/kv-cache-tier-persistence`

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

token-level compression [17, 9, 7, 8] differ in granularity. Cost-aware eviction is old: GreedyDual-Size [3] and its learned successors [14, 1] already evict by value per byte, and our V2 is GD-Size without aging. What differs is a *sign*: in web and CDN caching every hit is weakly positive, so admission is an optimization, whereas a sub-N^* hit here is net *negative*, which puts the threshold in provisioning and admission rather than in an eviction score.

2 Setup

What a tier holds, and why there is more than one. The unit of management is a whole conversation: one session's KV tensors form a single object of tens of MB to a few GB. When a session goes idle and VRAM is needed, a serving engine would discard it, forcing a full re-prefill on return; instead we serialize it one tier down and read it back later. The hierarchy exists because capacity and restore cost pull against each other: VRAM is fast and scarce, NVMe restores at 10 ms + 2 GB/s for far less per byte, object storage cheaper still at 100 ms + 500 MB/s. Demotion is triggered by capacity, not by a timer: when a tier is full, its eviction policy picks a victim, which moves down a level or leaves the hierarchy entirely from the last one. Each tier therefore has its *own* restore cost, which is what makes the metric fail: a hit from a slow tier can cost more to restore than the prefill it avoids, and hit rate counts it anyway.

Policies. Six policies replay identical traces, each evicting the entry minimizing its own score. For entry i, let v_i be the GPU time its cached tokens would save, s_i its bytes, $\mathbb{E}[\Delta t_i]$ its estimated time to next access, and f_i, r_i, n_i its access count, decayed idle time and normalized token count. The scores are: *LRU*, last access time; a *recency–frequency heuristic*, $0.4f_i + 0.4r_i + 0.2n_i$; *Logistic V1*, $\hat{P}(\text{resume} \le 1\text{h})$ from a nine-feature model (test AUC 0.70); *Value Density V2*, v_i/s_i; *V2+AC*, the same but admitting only entries above a value threshold; and *Space-Time V3*, $v_i/(s_i\,\mathbb{E}[\Delta t_i])$. V3 divides by expected lifetime so an entry is charged for the space-*time* it occupies, after LHD [1]. They differ sharply in what they consume: LRU needs only access order, the heuristic adds counts and sizes, V1 a trained model over nine features, V2/V3 the cost model pricing v_i. That widening dependence is what §5 shows makes them fragile.

Workloads and clock. Synthetic profiles use Poisson arrivals and log-normal token counts: enterprise (200/hr, μ=2048, σ=0.5) and power user (80/hr, μ=8192, σ=1.2), clamped at 8,192 tokens over a 6-hour window at 500 MB (50/150/300 MB across three tiers); a third casual profile never constrains the cache. Each user carries a persistent persona (return propensity, verbosity, a diurnal window); without one, per-user features are i.i.d. noise and no predictor beats chance. Traces replay on a *virtual clock*: wall-clock replay collapses session age, idle time, and hour-of-day to near zero, invalidating any policy that reasons about time while leaving order-based ones untouched.

Real traces. We also replay the Azure LLM inference trace [11, 15]: 27.3M production requests over one week, whose arrival process and context sizes (median 928, p95 4,943 tokens) transfer verbatim into ten 6-hour windows used as paired replicates. It cannot provide session identity, and linkage is not reconstructable: a greedy context-growth matcher links 87.8% of requests, and an indistinguishable 87.8% under a permutation control that destroys all true chains. We therefore impose return behavior with the same persona machinery, so differences from synthetic come from the arrival process and sizes, which are real. The learned policies run zero-shot.

Value accounting. Each hit is credited the prefill it avoids for the tokens *actually cached*, minus the modeled restore from its tier, both priced by Eq. 1; a hit whose restore exceeds its avoided prefill scores zero. We report GPU-seconds saved per day under both TinyLlama-1.1B (22 KB/token) and Llama-2-70B (320 KB/token, GQA) economics. The GPU-second is the scale-free quantity: dollars are a linear rescaling that depends on a reader's negotiated price, and at our arrival rates the dollar figures would say more about the size of the simulated deployment than about the policies.[2] The cost model is parameterized by the served architecture rather than fitted constants, and validated against hardware: over a $12\times$ context sweep on a T4 its predicted prefill tracks measurement within 1.15–$1.66\times$, and its break-even lengths match an independently coded analytic model exactly (App. A). Every cell runs 10 seeds with all policies on the identical per-seed trace, so comparisons are *paired*: per-seed deltas against LRU with 95% t-CIs.

[2] A sub-N^* hit is credited zero rather than the negative value it actually delivers. Crediting negatives is arguably more correct and would widen every gap we report, since LRU earns by far the most such hits; we floor at zero as the conservative choice.

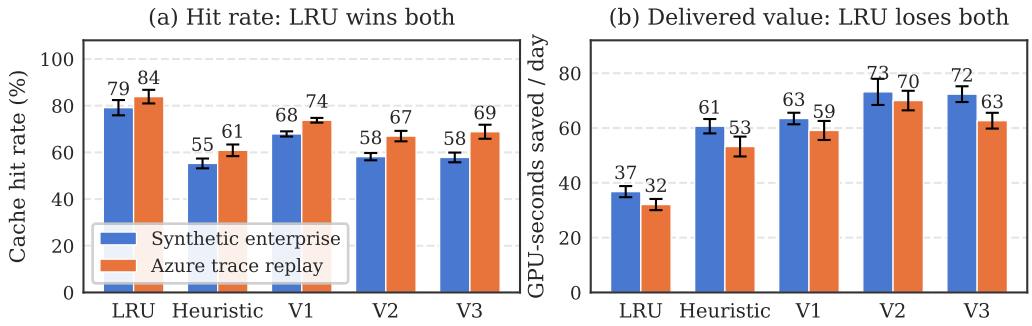


Figure 1: The same runs, two metrics: LRU wins hit rate on both workloads and delivers the least value on both. 10 paired seeds (synthetic) or ten 6-hour windows (Azure), 95% t-CIs, 500 MB.

3 Hit Rate and Value Reverse

LRU wins the metric we set out to beat, taking the highest hit rate on every constrained workload: 79.1% [75.9, 82.4] on enterprise against Logistic V1's 67.9% [66.8, 69.0], and 96.0% against 83.9% on power user.

That verdict is narrower than it sounds. On enterprise traffic (Figure 1) LRU simultaneously delivers the least value of any policy: V2 saves $1.99\times$ as much GPU time ($+36.4$ GPU-s/day $[+30.4, +42.4]$), V3 $1.97\times$, V1 $1.73\times$, and even the untrained heuristic $1.65\times$, every delta significant. Both replicate on real traffic, and the second strengthens: on the Azure replay LRU again wins hit rate (83.9%, by 16.9 points over V2) and again delivers the least value, V2 saving $2.18\times$. The inversion is *larger* on real traffic because real sizes are more skewed, so more mass falls below N^* (§4).

4 Mechanism: A Threshold and a Superlinearity

Restoring a persisted cache moves $O(N)$ bytes, while recomputing its prefill costs $O(N)$ weight FLOPs plus $O(N^2)$ attention:

$$t_{\text{restore}}(N) = \ell_{\text{tier}} + \frac{2LH_{kv}dbN}{B_{\text{tier}}}, \qquad t_{\text{prefill}}(N) = \frac{2PN + 4LH_q dN^2}{F\eta}, \tag{1}$$

for L layers, H_{kv} KV heads (GQA shrinks the cache, not the scores, so query heads H_q appear in prefill), head dimension d, b bytes/element, tier latency ℓ, bandwidth B, P parameters, peak F FLOP/s, and utilization η. Restore is linear and prefill superlinear, so they cross at a break-even length N^*, set by KV bytes per token over prefill FLOPs per token. That ratio spans four orders of magnitude: on an A100 at η=0.45, N^* is 1,561 tokens from NVMe (10 ms, 2 GB/s) and 25,901 from object storage (100 ms, 500 MB/s) for TinyLlama-1.1B; 43,559 and 254,203 for Llama-2-7B; and 12 and 289 for Llama-2-70B, whose GQA shrinks bytes moved while parameters inflate FLOPs avoided. The 7B figure is not a slip: a 512 KB/token MHA cache is so large against its prefill FLOPs that NVMe offload never repays at realistic contexts. That is why systems shipping KV state compress it [8] or use DRAM and RDMA, not a 2 GB/s disk [12]. Our η=0.45 is generous to prefill; higher MFU pushes N^* out, not in. The threshold is objective-agnostic too: the same inequality governs time-to-first-token.

Hit rate expresses neither effect. Under TinyLlama economics every cold-tier hit in our token range is worth exactly zero, and LRU, blind to size and tier alike, earns 812 of its 1,237 enterprise hits there; above N^*, meanwhile, savings grow superlinearly, so one 8k-token session can outweigh a dozen 2k-token ones.

Table 1 separates them two ways. Repricing at Llama-2-70B, where every hit clears break-even, collapses the enterprise gap to 1.07–$1.08\times$ and hands power-user traffic back to LRU; the threshold thus accounts for roughly $1.85\times$ and the magnitude effect for the residual $\sim 1.08\times$. Deleting the cold tier instead, at fixed architecture and 500 MB, estimates that residual independently at 1.13–$1.15\times$: V1 and V3 each deliver $1.15\times$ more than LRU while conceding 11 hit-rate points. Two

independent manipulations both leave a residual of roughly $1.1\times$, isolating the same superlinearity. Admission control, by contrast, never helps: V2+AC concedes 4.8 GPU-s/day to plain V2 on enterprise $[-5.3, -4.3]$ and 6.1 on the Azure replay $[-6.7, -5.4]$, because under sustained load the cache runs near-full and refusing entries at the door only costs cardinality (App. B).

The same table carries this paper's largest effect, and it is not a policy effect. *Deleting the cold tier raises LRU from 36.8 to 131.0 GPU-s/day, a factor of* 3.6*, at identical total capacity*; two-tier LRU beats the best three-tier policy we built by $1.8\times$. The same bytes provisioned differently beat the same bytes managed better, so the $1.99\times$ carries a condition: it holds for a deployment our own model advises against building, and the portable claim is the $1.15\times$ surviving correct provisioning, found only by pricing hits rather than counting them.

Ground truth: identical perfect hit rates, different value. Perhaps the learned policies lose because their predictor is weak (AUC 0.70), not because the objective is wrong. We replay the same traces with *oracle* policies reading the actual future, legitimate because replay is open-loop: *Bélády* [2], plus oracle V1 and V3 on ground-truth returns. All three cache perfectly at 100.0%, so every miss in §3 is a prediction failure, not a capacity necessity, and LRU trails them $5.70\times$ in value against 20.9 hit-rate points.

That setup yields the cleanest evidence here: every other source of difference is gone. Hit rate cannot distinguish Bélády from Oracle-V3: both read 100.0%, a paired difference of exactly zero on every seed. Their value differs significantly nonetheless, by $+2.9$ GPU-s/day $[+2.0, +3.8]$ on enterprise, because once nothing need ever be dropped the only decision left is *which tier* holds each entry. The effect is small (1.4%) but real, and hit rate reports it as exactly zero.

5 Discussion and Conclusion

Two bugs, both flattering, both ours. We corrected our own evaluation twice, in opposite directions: a wall-clock replay bug flattered the learned policy, an architecture-free cost model flattered LRU (App. C). The asymmetry is structural. A policy consuming only access order is insensitive to broken time, features, *and* value models alike; every richer policy is exposed to all three. Harness bugs therefore flatter whichever side depends on the broken component rather than perturbing rankings at random, so LRU comparisons controlling neither replay semantics nor cost-model fidelity deserve re-examination *both* ways.

A decision rule, in order of payoff. (1) Compute N^* per tier from Eq. 1: published geometry and storage constants, no measurement. (2) Do not build tiers whose N^* exceeds the workload's context lengths, the largest lever we measured at $3.6\times$. (3) Refuse admission below the surviving tiers' N^*, the step our data supports least directly, since density-thresholded admission never helped us and its payoff arrives as step (2). (4) Only then choose a policy, and never tune on hit rate before establishing the regime.

Limitations. Return behavior and value are both modeled rather than observed; Apps. A and D bound each. No policy we built wins *both* metrics anywhere.

Conclusion. Where a workload straddles N^*, hit rate ranks KV-cache policies backwards: it picks the worst one available and calls it best. Beating that metric does not require beating LRU.

Table 1: Two controls decomposing the effect (enterprise, paired seeds; heuristic omitted, see §3). **Middle**: repricing at Llama-2-70B moves N^* to 12 tokens, switching the threshold off, and the best margin over LRU falls from $1.99\times$ to $1.08\times$. **Right**: deleting the cold tier at fixed 500 MB leaves $1.15\times$.

	TinyLlama, 3-tier		Llama-2-70B, 3-tier		TinyLlama, 2-tier	
Policy	Hit%	GPU-s/d	Hit%	GPU-s/d	Hit%	GPU-s/d
LRU	**79.1**	36.8	**79.1**	8,173	**79.4**	131.0
Logistic V1	67.9	63.5	67.9	**8,806**	68.0	**150.7**
ValDensity V2	58.2	**73.2**	67.1	8,722	58.5	147.4
SpaceTime V3	57.8	72.4	63.2	8,783	58.1	150.0

References

[1] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. LHD: Improving cache hit rate by maximizing hit density. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018.

[2] László A. Bélády. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[3] Pei Cao and Sandy Irani. Cost-aware WWW proxy caching algorithms. In *Proceedings of the USENIX Symposium on Internet Technologies and Systems (USITS)*, 1997.

[4] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC)*, 2024.

[5] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the 7th Annual Conference on Machine Learning and Systems (MLSys)*, 2024.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[7] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[8] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.

[9] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[10] LMCache Team. LMCache: Accelerating LLM serving with reusable KV caches. GitHub Repository, 2024. URL `https://github.com/LMCache/LMCache`.

[11] Microsoft Azure. AzurePublicDataset: Azure LLM inference trace 2024. GitHub Repository, 2024. URL `https://github.com/Azure/AzurePublicDataset`.

[12] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025.

[13] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[14] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Béládys for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[15] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.

[16] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[17] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

A Cost-model validation

Every value figure in this paper is produced by a model, so the model is the thing most worth attacking. An earlier version of it deserved the attack: it priced prefill with two hardcoded constants and no model geometry, placing the linear/quadratic crossover near 1,300 tokens where real architectures cross between 12 and 254k, and predicting 57,747 ms for a 598-token TinyLlama prefill that a T4 measures at ∼37 ms. The model used here (Eq. 1) is parameterized by published model geometry and storage constants instead, and we check it two ways.

Against hardware. Table 2 sweeps context length on Llama-3.2-1B (16 layers, GQA 8/32, 32 KB/token) on a T4 and compares predicted prefill against measured cold TTFT. Predictions track measurement within 1.15–1.66× over a 12× range of context, and the residual is one-sided: measurement is consistently slower than prediction, implying an achieved MFU near 0.22 against the 0.30 assumed. The functional form is what matters for our argument, and it holds: over that range cold prefill grows 14.8× while the warm path grows 1.8×.

Against an independent implementation. The break-even lengths quoted in §4 are computed by `benchmarks/breakeven_analysis.py` from the same expressions written separately, and agree exactly with the simulator's cost model at all three architectures and both tiers. This catches transcription errors, not modeling ones.

Table 2: Predicted versus measured prefill, Llama-3.2-1B on a T4, five trials per length (medians). The warm path is tier load plus tensor restore plus one forward pass.

Tokens	Measured prefill	Predicted	Ratio	Measured warm	Predicted restore
512	111 ms	67 ms	1.66×	101 ms	18 ms
1,024	157 ms	137 ms	1.15×	115 ms	27 ms
2,048	340 ms	288 ms	1.18×	115 ms	44 ms
4,096	878 ms	632 ms	1.39×	146 ms	77 ms
6,144	1,645 ms	1,032 ms	1.59×	181 ms	111 ms

The fixed cost of the measured warm path is the one constant that does *not* transfer: fitting it gives 94.5 ms of overhead plus 13.4 μs per token, an implied 2.44 GB/s against the 2.0 GB/s the model assumes. The bandwidth transfers; the 94.5 ms does not, because a Colab ephemeral disk read through `torch.load` is not an NVMe read. N^* must therefore be computed from the storage path actually deployed.

B Admission control

Value Density V2+AC is V2 plus a value-density admission threshold, derived per run rather than hardcoded. It never helped on any workload we ran (Table 3), and the reason is visible in the hit rates: it declines entries at the door without freeing capacity that the eviction policy would not have freed anyway, so it pays cardinality for nothing. Under sustained arrivals the cache sits at near-full utilization regardless.

This is a result about *density*-thresholded admission, and it should not be read as evidence against the N^*-thresholded admission of step (3) in §5, which is a different rule and which we do not evaluate as a standalone policy. It is also conditioned on our accounting flooring sub-N^* hits at zero: a policy that merely declines a worthless hit cannot gain under that floor, and would gain under an accounting that charged the restore.

Table 3: Admission control against plain value density, 10 paired replicates each. Deltas are per-replicate against V2 with 95% t-CIs.

Workload	Policy	Hit%	GPU-s/day	Δ vs V2 [CI]
Synthetic enterprise	ValDensity V2	58.2	**73.2**	—
	V2+AC	58.2	68.4	$-4.8\,[-5.3, -4.3]$
Azure replay	ValDensity V2	67.0	**70.0**	—
	V2+AC	66.8	63.9	$-6.1\,[-6.7, -5.4]$

C The two evaluation defects

Both defects were ours, both flattered a different side, and neither perturbed rankings at random. We report them because the pattern is the transferable part.

Wall-clock replay (flattered the learned policy). An early harness replayed a 6-hour trace at wall-clock speed and credited hits with unclamped resume-time token counts. Collapsing simulated time drives session age, idle time, and hour-of-day to approximately zero, which zeroes exactly the features the predictor was trained on, while the accounting credited a different quantity than the policies optimized. It showed a learned policy beating LRU. Fixing it reversed the result, and we reported LRU winning both metrics for some time afterwards.

An architecture-free cost model (flattered LRU). The simulator priced prefill with two hardcoded constants, no model geometry at all, placing the linear/quadratic crossover near 1,300 tokens where real architectures cross between 12 and 254k. This is not a rescaling, so the usual invariance argument, that a constant factor cannot change rankings, does not apply: the error inflated an 8k-token session's worth relative to a 512-token one by 3–$4.5\times$, exaggerating precisely the effect the study measures. Re-pricing from architecture inverted the value comparison, left hit rates untouched, and retracted a "static value density collapses" finding we had previously reported, which had been a property of the cost model rather than of the objective.

We caught the second only when a review of the cost model compared its 57.7-second prediction for a 598-token prefill with the $\sim$37 ms the hardware measured. No test in our suite would have caught it, because none existed; App. A is the test that does now.

D Persona strength

Return behavior in our workloads is generated, not observed, and the dispersion σ of per-user return propensity bounds how much structure any predictor can recover. We sweep it, retraining the predictor at each setting so the model is as good as that world allows, and re-run the enterprise matrix (Table 4).

Table 4: Persona strength sweep (enterprise, 5 paired seeds, TinyLlama economics). Value is GPU-seconds saved per day, as a ratio to LRU.

σ	Predictor AUC	LRU hit%	V1 ×LRU	V3 ×LRU
0.0	0.659	95.3	0.90	1.34
0.3	0.670	89.5	1.47	1.50
0.6	0.700	80.3	1.66	1.91
1.0	0.739	66.3	1.79	2.25

The two claims separate cleanly. The *learned* policy's standing depends on σ, as it must: at $\sigma{=}0$, where users are behaviorally identical, V1 falls to $0.90\times$ LRU and the difference is not significant. A classifier whose premise is per-user prediction should fail when there is no per-user structure, and it does. The *metric* finding does not depend on σ: V3 beats LRU on value at every setting, and the gap is smallest exactly where LRU is strongest, which is the regime least favorable to our argument and still not favorable enough to overturn it.